



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт Информационных Технологий
Кафедра Вычислительной Техники (ВТ)

ОТЧЁТ ПО ПРАКТИЧЕСКИМ РАБОТАМ

по дисциплине

«Проектирование и обучение нейронных сетей»

Выполнил студент группы
ИКБО-42-23

Голев С.С.

Принял старший преподаватель
кафедры ВТ

Семенов Р.Э.

Практическая работа выполнена

«__»_____2025 г.

«Зачтено»

«__»_____2025 г.

Москва 2025 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1. Обучение по правилам Хебба.....	5
1.1 Постановка задачи.....	5
1.2 Результат работы.....	5
2. Дельта правило.....	6
2.1 Постановка задачи.....	6
2.2 Результат работы.....	6
3. Обратное распространение ошибки.....	7
3.1 Постановка задачи.....	7
3.2 Результат работы.....	7
4. Радиально-базисных функций.....	8
4.1 Постановка задачи.....	8
4.2 Результат работы.....	8
5. Карты Кохонена.....	9
5.1 Постановка задачи.....	9
5.2 Результат работы.....	9
6. Встречного распространения.....	10
6.1 Постановка задачи.....	10
6.2 Результат работы.....	10
7. Рекуррентные сети.....	11
7.1 Постановка задачи.....	11
7.2 Результат работы.....	11
8. Сверточные сети.....	12
8.1 Постановка задачи.....	12
8.2 Результат работы.....	12

ЗАКЛЮЧЕНИЕ.....	13
СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ.....	15
ПРИЛОЖЕНИЕ.....	16

ВВЕДЕНИЕ

Современное развитие информационных технологий и стремительный рост объёма данных привели к широкому распространению искусственных нейронных сетей как одного из наиболее эффективных инструментов анализа, обработки и интерпретации информации. Нейросетевые модели успешно применяются в задачах классификации, прогнозирования, оптимизации, распознавания образов и во многих других областях науки и промышленности. Для успешного построения и применения таких систем необходимо глубокое понимание принципов функционирования различных архитектур нейронных сетей, а также методов их обучения.

Целью выполнения данного комплекса лабораторных работ является изучение базовых и современных подходов к проектированию и обучению нейронных сетей. В ходе работы рассматриваются ключевые алгоритмы обучения и типы сетевых архитектур, позволяющие решать широкий спектр задач машинного обучения.

В процессе выполнения лабораторных работ будут реализованы и исследованы следующие типы нейронных сетей и алгоритмов обучения:

1. Нейронные сети, обучаемые по правилам Хебба
2. Нейронные сети с обучением по дельта-правилу
3. Алгоритм обратного распространения ошибки
4. Сети на основе радиально-базисных функций (RBF-сети)
5. Самоорганизующиеся карты Кохонена
6. Сети с встречным распространением сигнала
7. Рекуррентные нейронные сети
8. Сверточные нейронные сети

Изучение перечисленных моделей позволит понять различия между методами обучения с учителем, без учителя и гибридными подходами, а также получить практические навыки построения и анализа нейросетевых систем.

Таким образом, данная работа направлена на формирование теоретической базы и приобретение практического опыта по разработке, обучению и применению нейронных сетей в решении прикладных задач.

1. Обучение по правилам Хебба

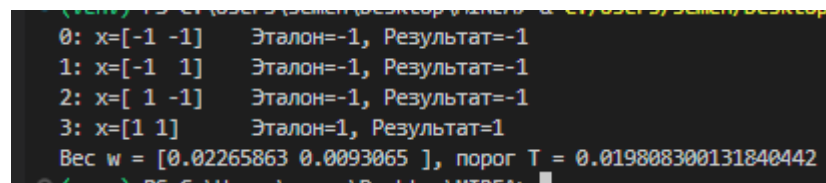
1.1 Постановка задачи

Задача состоит в построении простейшей однослойной сети, демонстрирующей корреляционный принцип обучения «нейроны, активирующиеся вместе, усиливают свои связи». Необходимо обучить её решать задачу реализации логической функции И на двух входах.

После обучения должны быть определены устойчивые значения весовых коэффициентов. На их основе создаётся новая модель, которая тестируется на всех входах из таблицы истинности, с целью подтверждения или тестирования способности правила Хебба решать задачу классификации.

1.2 Результат работы

Реализуем данную нейронную сеть на языке Python и проверим её работоспособность (Рисунок 1.1). Сам код программы представлен в приложении А.



```
0: x=[-1 -1]   Эталон=-1, Результат=-1
1: x=[-1  1]   Эталон=-1, Результат=-1
2: x=[ 1 -1]   Эталон=-1, Результат=-1
3: x=[1  1]    Эталон=1, Результат=1
Вес w = [0.02265863 0.0093065 ], порог T = 0.019808300131840442
```

Рисунок 1.1 – Реализация обучения по правилам Хебба

2. Дельта правило

2.1 Постановка задачи

Необходимо построить перцептрон с обучением с учителем, способный реализовывать логическую операцию И. В процессе обучения корректируются веса на основе ошибки по дельта-правилу (правило Видроу — Хоффа).

После обучения итоговые веса фиксируются и используются для построения независимой модели, которую требуется протестировать на полной таблице входов и проанализировать полученные ошибки.

2.2 Результат работы

Реализуем данную нейронную сеть на языке Python и проверим её работоспособность (Рисунок 2.1). Сам код программы представлен в приложении Б.

```
Вес w = [0.02381899 0.01121283], порог T = 0.039332613782388226  
Вес w = [396.02381899 396.01121283], порог T = 0.03933261378238817  
Эталон: [-1 -1 -1 1]  
Выходы: [-1 -1 -1 1]
```

Рисунок 2.1 – Реализация обучения по дельта правилу

3. Обратное распространение ошибки

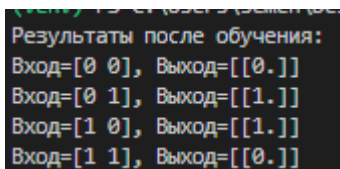
3.1 Постановка задачи

Требуется разработать и обучить многослойную нейронную сеть (минимум один скрытый слой) для решения задачи XOR, являющейся линейно-неразделимой.

Обучение проводится методом backpropagation. После завершения итераций оптимизации веса всех слоёв сохраняются и используются для создания итоговой модели. Выполняется полное тестирование для подтверждения способности сети к аппроксимации нелинейных зависимостей.

3.2 Результат работы

Реализуем данную нейронную сеть на языке Python и проверим её работоспособность (Рисунок 3.1). Сам код программы представлен в приложении В.



```
Результаты после обучения:  
Вход=[0 0], Выход=[[0.]]  
Вход=[0 1], Выход=[[1.]]  
Вход=[1 0], Выход=[[1.]]  
Вход=[1 1], Выход=[[0.]]
```

Рисунок 3.1 – Реализация обучения методом обратного распространения

4. Радиально-базисных функций

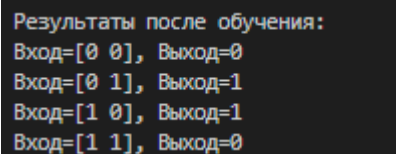
4.1 Постановка задачи

Цель — построить RBF-сеть с радиальными нейронами в скрытом слое для точной аппроксимации оператора XOR. Требуется определить центры и ширины радиальных функций, а также веса выходных нейронов.

После обучения необходимо построить итоговую модель по найденным параметрам и подтвердить корректность классификации входных данных.

4.2 Результат работы

Реализуем данную нейронную сеть на языке Python и проверим её работоспособность (Рисунок 4.1). Сам код программы представлен в приложении Г.



```
Результаты после обучения:  
Вход=[0 0], Выход=0  
Вход=[0 1], Выход=1  
Вход=[1 0], Выход=1  
Вход=[1 1], Выход=0
```

Рисунок 4.1 – Реализация обучения методом радиально-базисных функций

5. Карты Кохонена

5.1 Постановка задачи

Задача — применить метод конкурентного обучения без учителя для организации входных данных, на карте Кохонена.

В процессе обучения формируются веса, которые определяют расположение кластеров на карте. Эти веса фиксируются и используются для построения итоговой SOM, после чего осуществляется тестирование для оценки кластеризующих свойств.

5.2 Результат работы

Реализуем данную нейронную сеть на языке Python и проверим её работоспособность (Рисунок 5.1). Сам код программы представлен в приложении Д.

```
Расположение нейронов-победителей:  
Вход [0.2 0.1] → нейрон (np.int64(2), np.int64(0))  
Вход [0.1 0.2] → нейрон (np.int64(2), np.int64(0))  
Вход [0.9 0.8] → нейрон (np.int64(0), np.int64(2))  
Вход [0.9 0.9] → нейрон (np.int64(0), np.int64(2))
```

Рисунок 5.1 – Реализация обучения для кластеризации данных по картам Кохонена

6. Встречного распространения

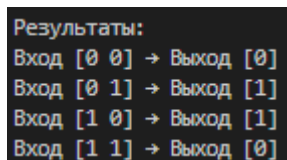
6.1 Постановка задачи

Требуется реализовать сеть, объединяющую принципы самоорганизации (Кохонен) и обучения с учителем (Гроссберг) для выполнения задачи XOR. В ходе обучения необходимо получить две матрицы весов — конкурирующего слоя и ассоциативного слоя.

После фиксации всех параметров строится окончательная модель, которая проверяется на способность восстанавливать правильные выходные значения для всех входных комбинаций.

6.2 Результат работы

Реализуем данную нейронную сеть на языке Python и проверим её работоспособность (Рисунок 6.1). Сам код программы представлен в приложении Е.



```
Результаты:  
Вход [0 0] → Выход [0]  
Вход [0 1] → Выход [1]  
Вход [1 0] → Выход [1]  
Вход [1 1] → Выход [0]
```

Рисунок 6.1 – Реализация обучения методом встречного распространения

7. Рекуррентные сети

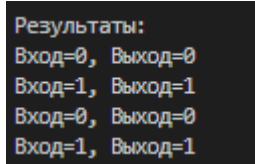
7.1 Постановка задачи

Цель — исследовать возможность использования внутренней памяти рекуррентной сети для реализации и тестирования данного типа нейронной сети.

После обучения сохраняются веса рекуррентных и выходных слоёв, формируется итоговая модель и проводится тестирование на новых последовательностях для проверки способности к запоминанию.

7.2 Результат работы

Реализуем данную нейронную сеть на языке Python и проверим её работоспособность (Рисунок 7.1). Сам код программы представлен в приложении Ж.



```
Результаты:  
Вход=0, Выход=0  
Вход=1, Выход=1  
Вход=0, Выход=0  
Вход=1, Выход=1
```

Рисунок 7.1 – Реализация обучения рекуррентной сети

8. Сверточные сети

8.1 Постановка задачи

Необходимо применить сверточную архитектуру для распознавания, как тема была выбрана тема «Выявление катаракты глаза».

8.2 Результат работы

Реализуем данную нейронную сеть на языке Python и проверим её работоспособность (Рисунок 8.1). Сам код программы представлен в приложении 3.

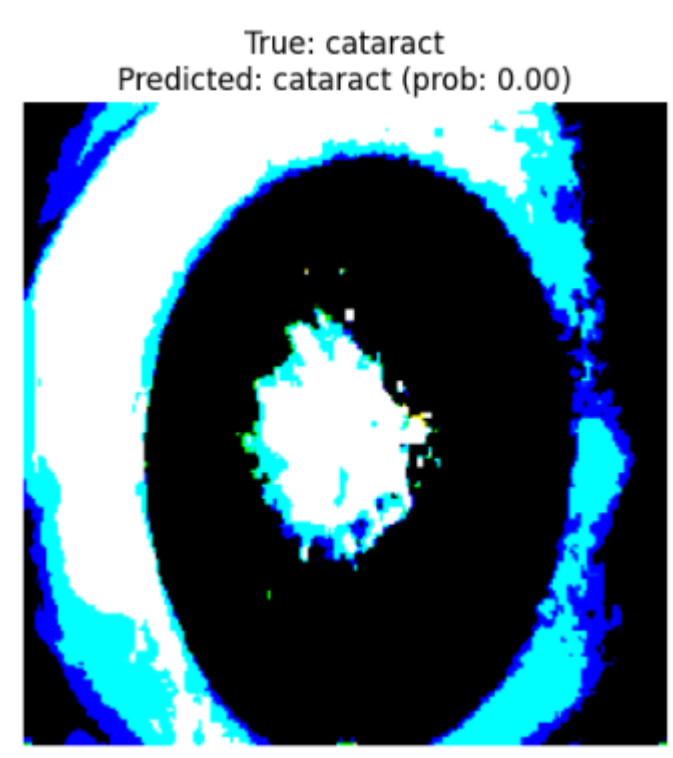


Рисунок 8.1 – Реализация сверточной нейронной сети для определения катаракты глаза

ЗАКЛЮЧЕНИЕ

В ходе выполнения комплекса лабораторных работ было проведено исследование различных типов нейронных сетей и алгоритмов их обучения. Рассмотренные модели охватывают как простейшие однослойные структуры, так и сложные многослойные и самоорганизующиеся архитектуры, что позволило получить целостное представление о современных методах проектирования нейронных сетей.

В рамках работ были реализованы сети, обучаемые по правилам Хебба и дельта-правилу, продемонстрировавшие возможность решения линейно-разделимых задач, таких как логическая функция И. Это подтвердило теоретические положения о применимости однослойных моделей к простым задачам классификации.

Использование алгоритма обратного распространения ошибки, радиально-базисных сетей, а также сверточных и рекуррентных архитектур позволило успешно решить задачу логической функции XOR, которая является нелинейной. Эти эксперименты показали важность скрытых слоёв и нелинейных преобразований для аппроксимации сложных зависимостей.

Также были изучены модели обучения без учителя — карты Кохонена и сеть встречного распространения, которые продемонстрировали возможности самоорганизации, кластеризации данных и ассоциативного запоминания.

Для каждой модели были определены итоговые весовые коэффициенты, на основе которых строились тестовые модели. Проверка показала корректность обучения и работоспособность полученных решений.

Таким образом, в ходе работы были достигнуты все поставленные цели:

- изучены принципы функционирования и обучения различных нейронных сетей;
- разработаны и протестированы модели для решения задач различной сложности;

- подтверждена эффективность рассмотренных методов на практических примерах.

Полученные результаты формируют фундаментальные практические навыки для дальнейшего углубленного изучения технологий искусственного интеллекта и машинного обучения.

СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ

Goodfellow I., Bengio Y., Courville A. Deep Learning. — Cambridge, MA : MIT Press, 2016. — 775 p.

Хайкин С. Нейронные сети: полный курс. 2-е изд. — Москва : Вильямс, 2006. — 1104 с.

Рашка С., Мирджалили В. Python и машинное обучение. — Москва : ДМК Пресс, 2018. — 520 с.

Bishop C. M. Neural Networks for Pattern Recognition. — Oxford : Oxford University Press, 1995. — 504 p.

Kohonen T. Self-Organizing Maps. 3rd ed. — Berlin : Springer, 2001. — 521 p.

Patterson J., Gibson A. Deep Learning: A Practitioner's Approach. — Beijing; Cambridge : O'Reilly Media, 2017. — 532 p.

Курдюмов В. Г., Терехов А. Н. Нейросетевые технологии. Учебное пособие. — Санкт-Петербург : Питер, 2020. — 304 с.

ПРИЛОЖЕНИЕ

Приложение А – Обучение по правилу Хэбба

Приложение Б – Дельта-правило

Приложение В – Обратное распространение ошибки

Приложение Г – Радиально-базисные функции (RBF-сеть)

Приложение Д – Карты Кохонена

Приложение Е – Сеть встречного распространения

Приложение Ж – Рекуррентная нейронная сеть (RNN)

Приложение З – Сверточная нейронная сеть (CNN)

Приложение А

Код нейронной сети основанной на обучении по правилу Хэбба.

Листинг А.1 — Основной алгоритм программы

```
import numpy as np

class Rosenblatt:
    def __init__(self, n_inputs, init_scale=0.05):
        self.w = np.random.uniform(0, init_scale, size=n_inputs)
        self.T = np.random.uniform(0, init_scale)

    def activation(self, x):
        net = np.dot(self.w, x) - self.T
        return 1 if net >= 0 else -1

    def modification(self, x, etal):
        self.w = self.w + x * etal
        self.T = self.T - etal

    def train(self, X, D):
        n_samples = X.shape[0]

        for i in range(n_samples):
            x = X[i]
            etal = D[i]
            y = self.activation(x)
            print(f"{i}: x={x}\tЭталон={etal}, Результат={y}")
            while y != etal:
                self.modification(x, etal)
                print(f"\tНовые значения -> w={self.w},
T={self.T}")

                y = self.activation(x)
                print(f"{i}: x={x}\tЭталон={etal},
Результат={y}")

if __name__ == "__main__":
    X = np.array([[-1, -1],
                  [-1, 1],
                  [ 1, -1],
                  [ 1, 1]])
    D = np.array([-1, -1, -1, 1])

    neuron = Rosenblatt(n_inputs=2)

    neuron.train(X, D)
    print(f"Вес w = {neuron.w}, порог T = {neuron.T}")
```

Приложение Б

Код нейронной сети, обучаемой по дельта-правилу.

Листинг Б.1 — Основной алгоритм программы

```
import numpy as np

class Delta:
    def __init__(self, n_inputs, init_scale=0.05):
        self.w = np.random.uniform(0, init_scale, size=n_inputs)
        self.T = np.random.uniform(0, init_scale)
        self.eta = 1
        self.err = 0

    def activation(self, x):
        S = np.dot(x, self.w)
        return -1 if S > self.T else 1
    def modification(self, x, e):
        self.w = self.w + self.eta * x * e
        self.T = self.T + self.eta * e
    def predict(self, X):
        S = X.dot(self.w)
        return np.where(S > self.T, 1, -1)
    def train(self, X, D):
        for step in range(1, 100):
            total_error = 0
            for i in range(X.shape[0]):
                x = X[i]
                d = D[i]
                y = self.activation(x)
                e = d - y
                total_error += e
                self.modification(x, e)
            if total_error < self.err:
                break

if __name__ == "__main__":
    X = np.array([[ -1, -1],
                  [ -1,  1],
                  [  1, -1],
                  [  1,  1]])
    D = np.array([ -1, -1, -1,  1])
    neuron = Delta(n_inputs=2)
    print(f"Вес w = {neuron.w}, порог T = {neuron.T}")
    neuron.train(X, D)
    print(f"Вес w = {neuron.w}, порог T = {neuron.T}")
    y_pred = neuron.predict(X)
    print("Эталон:", D)
    print("Выходы:", y_pred)
```

Приложение В

Код многослойного перцептрона с обучением методом обратного распространения ошибки.

Листинг В.1 — Основной алгоритм программы

```
import numpy as np

class BackpropagationNN:
    def __init__(self, n_inputs, n_hidden, n_outputs,
init_scale=0.5, alpha=0.5):
        self.alpha = alpha

        self.W1 = np.random.uniform(-init_scale, init_scale,
(n_inputs, n_hidden))
        self.b1 = np.random.uniform(-init_scale, init_scale, (1,
n_hidden))

        self.W2 = np.random.uniform(-init_scale, init_scale,
(n_hidden, n_outputs))
        self.b2 = np.random.uniform(-init_scale, init_scale, (1,
n_outputs))

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def sigmoid_derivative(self, y):
        return y * (1 - y)

    def forward(self, X):
        self.hidden_input = np.dot(X, self.W1) + self.b1
        self.hidden_output = self.sigmoid(self.hidden_input)

        self.final_input = np.dot(self.hidden_output, self.W2) +
self.b2
        self.final_output = self.sigmoid(self.final_input)

        return self.final_output

    def backward(self, X, D):
        error = D - self.final_output
        delta_output = error *
self.sigmoid_derivative(self.final_output)

        delta_hidden = delta_output.dot(self.W2.T) *
self.sigmoid_derivative(self.hidden_output)
```

Листинг В.2 — Продолжение листинга В.1

```
self.W2 += self.alpha * self.hidden_output.T.dot(delta_output)
self.b2 += self.alpha * np.sum(delta_output, axis=0,
keepdims=True)

self.W1 += self.alpha * X.T.dot(delta_hidden)
self.b1 += self.alpha * np.sum(delta_hidden, axis=0,
keepdims=True)

return np.mean(np.square(error))

def train(self, X, D, epochs=1000):
    for epoch in range(epochs):
        self.forward(X)
        loss = self.backward(X, D)

def predict(self, X):
    return self.forward(X)

if __name__ == "__main__":
    X = np.array([[0,0],
                  [0,1],
                  [1,0],
                  [1,1]])
    D = np.array([[0],[1],[1],[0]])

    neuron = BackpropagationNN(n_inputs=2, n_hidden=2,
n_outputs=1, alpha=0.5)
    neuron.train(X, D, epochs=5000)

    print("Результаты после обучения:")
    for i, x in enumerate(X):
        print(f"Вход={x},
Выход={neuron.predict(x.reshape(1,-1)).round(0)}")
```

Приложение Г

Код нейронной сети на основе радиально-базисных функций.

Листинг Г.1 — Основной алгоритм программы

```
import numpy as np

class RBFNetwork:
    def __init__(self, n_inputs, n_hidden, n_outputs, alpha=0.1):
        self.alpha = alpha

        self.centers = np.random.rand(n_hidden, n_inputs)
        self.sigma = np.ones(n_hidden)
        self.W = np.random.randn(n_hidden, n_outputs)
        self.b = np.random.randn(1, n_outputs)

    def _gaussian(self, x, c, s):
        return np.exp(-np.linalg.norm(x - c) ** 2 / (2 * s ** 2))

    def _rbf_layer(self, X):
        G = np.zeros((X.shape[0], self.centers.shape[0]))
        for i, x in enumerate(X):
            for j, c in enumerate(self.centers):
                G[i, j] = self._gaussian(x, c, self.sigma[j])
        return G

    def _sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def forward(self, X):
        G = self._rbf_layer(X)
        output_linear = np.dot(G, self.W) + self.b
        output = self._sigmoid(output_linear)
        return output, G

    def train(self, X, D, epochs=200):
        random_idx = np.random.choice(X.shape[0],
self.centers.shape[0], replace=False)
        self.centers = X[random_idx]

        d_max = np.max([np.linalg.norm(c1 - c2) for c1 in
self.centers for c2 in self.centers])
        self.sigma[:] = d_max / np.sqrt(2 *
self.centers.shape[0])
```

```
for epoch in range(epochs):
    Y, G = self.forward(X)
    error = D - Y
    loss = np.mean(np.square(error))
    self.W += self.alpha * np.dot(G.T, error)
    self.b += self.alpha * np.sum(error, axis=0,
keepdims=True)

    def predict(self, X):
        Y, _ = self.forward(X)
        return (Y >= 0.5).astype(int)

if __name__ == "__main__":
    X = np.array([[0, 0],
                  [0, 1],
                  [1, 0],
                  [1, 1]])
    D = np.array([[0], [1], [1], [0]])

    rbf = RBFNetwork(n_inputs=2, n_hidden=4, n_outputs=1,
alpha=0.5)
    rbf.train(X, D, epochs=200)

    print("\nРезультаты после обучения:")
    for x in X:
        y = rbf.predict(x.reshape(1, -1))
        print(f"Вход={x}, Выход={y[0][0]}")
```

Приложение Д

Код самоорганизующейся карты Кохонена.

Листинг Д.1 — Основной алгоритм программы

```
import numpy as np

class KohonenSOM:
    def __init__(self, n_inputs, grid_size=(3, 3), alpha=0.5,
sigma=None):
        self.n_inputs = n_inputs
        self.grid_size = grid_size
        self.alpha = alpha
        self.sigma = sigma if sigma is not None else
max(grid_size) / 2

        self.weights = np.random.rand(grid_size[0], grid_size[1],
n_inputs)

        self.neuron_locations = np.array(
            [[i, j] for i in range(grid_size[0]) for j in
range(grid_size[1])])

    def _find_bmu(self, x):
        distances = np.linalg.norm(self.weights - x, axis=2)
        bmu_idx = np.unravel_index(np.argmin(distances),
distances.shape)
        return bmu_idx

    def _neighborhood_function(self, bmu_idx, iteration,
max_iter):
        decayed_sigma = self.sigma * np.exp(-iteration /
(max_iter / np.log(self.sigma + 1e-8)))
        bmu_loc = np.array(bmu_idx)

        dist_sq = np.sum((self.neuron_locations - bmu_loc) ** 2,
axis=1)
        h = np.exp(-dist_sq / (2 * decayed_sigma ** 2))
        return h.reshape(self.grid_size)
```

Листинг Д.2 — Продолжение листинга Д.1

```
def train(self, X, epochs=100):
    for epoch in range(epochs):
        alpha_t = self.alpha * np.exp(-epoch / epochs)

        for x in X:
            bmu_idx = self._find_bmu(x)
            h = self._neighborhood_function(bmu_idx, epoch,
epochs)
            for i in range(self.grid_size[0]):
                for j in range(self.grid_size[1]):
                    self.weights[i, j, :] += alpha_t * h[i,
j] * (x - self.weights[i, j, :])

    def predict(self, x):
        return self._find_bmu(x)

if __name__ == "__main__":
    X = np.array([
        [0.2, 0.1],
        [0.1, 0.2],
        [0.9, 0.8],
        [0.9, 0.9]
    ])

    som = KohonenSOM(n_inputs=2, grid_size=(3, 3), alpha=0.6)
    som.train(X, epochs=50)

    print("\nРасположение нейронов-победителей:")
    for x in X:
        print(f"Вход {x} → нейрон {som.predict(x)}")
```

Приложение Е

Код нейронной сети встречного распространения сигнала.

Листинг Е.1 — Основной алгоритм программы

```
import numpy as np

class CounterPropagationNetwork:
    def __init__(self, n_inputs, n_hidden, n_outputs, alpha=0.5,
beta=0.3):
        self.n_inputs = n_inputs
        self.n_hidden = n_hidden
        self.n_outputs = n_outputs

        self.alpha = alpha
        self.beta = beta

        self.W_kohonen = np.random.rand(n_hidden, n_inputs)
        self.W_kohonen = self.W_kohonen /
np.linalg.norm(self.W_kohonen, axis=1, keepdims=True)

        self.W_grossberg = np.random.rand(n_hidden, n_outputs)

    def normalize(self, x):
        norm = np.linalg.norm(x)
        return x / norm if norm != 0 else x

    def kohonen_winner(self, x):
        activations = np.dot(self.W_kohonen, x)
        return np.argmax(activations)

    def train(self, X, D, epochs=100):
        for epoch in range(epochs):
            for x, d in zip(X, D):
                x = self.normalize(x)
                winner = self.kohonen_winner(x)
                self.W_kohonen[winner] += self.alpha * (x -
self.W_kohonen[winner])
                self.W_kohonen[winner] /=
np.linalg.norm(self.W_kohonen[winner])
                self.W_grossberg[winner] += self.beta * (d -
self.W_grossberg[winner])

    def predict(self, x):
        x = self.normalize(x)
        winner = self.kohonen_winner(x)
        y = self.W_grossberg[winner]
        return np.where(y >= 0.5, 1, 0)
```

Листинг E.2 — Продолжение листинга E.1

```
if __name__ == "__main__":
    X = np.array([
        [0, 0],
        [0, 1],
        [1, 0],
        [1, 1]
    ])
    D = np.array([
        [0],
        [1],
        [1],
        [0]
    ])

    net = CounterPropagationNetwork(n_inputs=2, n_hidden=4,
n_outputs=1, alpha=0.4, beta=0.3)
    net.train(X, D, epochs=100)

    print("\nРезультаты:")
    for x in X:
        print(f"Вход {x} → Выход {net.predict(x)}")
```

Приложение Ж

Код рекуррентной нейронной сети.

Листинг Ж.1 — Основной алгоритм программы

```
import numpy as np

class ElmanRNN:
    def __init__(self, n_inputs, n_hidden, n_outputs, alpha=0.1):
        self.n_inputs = n_inputs
        self.n_hidden = n_hidden
        self.n_outputs = n_outputs
        self.alpha = alpha

        self.U = np.random.uniform(-0.5, 0.5, (n_hidden,
n_inputs))
        self.W = np.random.uniform(-0.5, 0.5, (n_hidden,
n_hidden))
        self.V = np.random.uniform(-0.5, 0.5, (n_outputs,
n_hidden))

        self.bh = np.zeros((n_hidden, 1))
        self.by = np.zeros((n_outputs, 1))

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def dsigmoid(self, y):
        return y * (1 - y)

    def tanh(self, x):
        return np.tanh(x)

    def dtanh(self, y):
        return 1 - y ** 2

    def forward(self, X):
        T = len(X)
        self.h, self.y = {}, {}
        self.h[-1] = np.zeros((self.n_hidden, 1))
        for t in range(T):
            x_t = X[t].reshape(-1, 1)
            self.h[t] = self.tanh(np.dot(self.U, x_t) +
np.dot(self.W, self.h[t-1]) + self.bh)
            self.y[t] = self.sigmoid(np.dot(self.V, self.h[t]) +
self.by)
        return self.y, self.h
```

```

def backward(self, X, D):
    T = len(X)
    dU = np.zeros_like(self.U)
    dW = np.zeros_like(self.W)
    dV = np.zeros_like(self.V)
    dbh = np.zeros_like(self.bh)
    dby = np.zeros_like(self.by)
    dh_next = np.zeros((self.n_hidden, 1))
    for t in reversed(range(T)):
        x_t = X[t].reshape(-1, 1)
        d = D[t].reshape(-1, 1)
        dy = (self.y[t] - d) * self.dsigmoid(self.y[t])
        dV += np.dot(dy, self.h[t].T)
        dby += dy
        dh = np.dot(self.V.T, dy) + dh_next
        dh_raw = dh * self.dtanh(self.h[t])
        dbh += dh_raw
        dU += np.dot(dh_raw, x_t.T)
        dW += np.dot(dh_raw, self.h[t-1].T)
        dh_next = np.dot(self.W.T, dh_raw)
    for param, dparam in zip([self.U, self.W, self.V,
self.bh, self.by],
                                [dU, dW, dV, dbh, dby]):
        param -= self.alpha * dparam
def train(self, X, D, epochs=1000):
    for epoch in range(epochs):
        self.forward(X)
        self.backward(X, D)
        if epoch % 100 == 0:
            loss = np.mean([(D[t] - self.y[t])**2 for t in
range(len(X))])
    def predict(self, X):
        y_pred, _ = self.forward(X)
        outputs = [1 if y[0] >= 0.5 else 0 for y in [v.flatten()
for v in y_pred.values()]]
        return outputs
if __name__ == "__main__":
    X = [np.array([0]), np.array([1]), np.array([0]),
np.array([1])]
    D = [np.array([0]), np.array([1]), np.array([0]),
np.array([1])]
    rnn = ElmanRNN(n_inputs=1, n_hidden=3, n_outputs=1,
alpha=0.05)
    rnn.train(X, D, epochs=1000)
    print("\nРезультаты:")
    Y = rnn.predict(X)
    for i, x in enumerate(X):
        print(f"Вход={x[0]}, Выход={Y[i]}")

```

Приложение 3

Код сверточной нейронной сети.

Листинг 3.1 — Основной алгоритм программы

```
import tensorflow as tf
import tensorflow as tf
import time
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D,
Flatten, Dense
from tensorflow.keras.preprocessing.image import
ImageDataGenerator
from tensorflow.keras.applications.resnet import preprocess_input
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import classification_report,
confusion_matrix
import seaborn as sns
import os

TRAIN_DIR = 'data/train'
TEST_DIR = 'data/test'
IMG_SIZE = (224, 224)
BATCH_SIZE = 16

print("Структура train:")
for cls in os.listdir(TRAIN_DIR):
    cls_path = os.path.join(TRAIN_DIR, cls)
    print(f"- {cls}: {len(os.listdir(cls_path))} изображений")

print("\nСтруктура test:")
for cls in os.listdir(TEST_DIR):
    cls_path = os.path.join(TEST_DIR, cls)
    print(f"- {cls}: {len(os.listdir(cls_path))} изображений")
train_datagen_resnet = ImageDataGenerator(
    preprocessing_function=preprocess_input,
    rotation_range=30,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest',
    brightness_range=[0.8, 1.2])
```

Листинг 3.2 — Продолжение листинга 3.1

```
test_datagen_resnet = ImageDataGenerator(
    preprocessing_function=preprocess_input
)
train_generator_resnet =
train_datagen_resnet.flow_from_directory(
    TRAIN_DIR,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='binary',
    shuffle=True
)

test_generator_resnet = test_datagen_resnet.flow_from_directory(
    TEST_DIR,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='binary',
    shuffle=False
)

train_datagen_cnn = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    horizontal_flip=True
)

test_datagen_cnn = ImageDataGenerator(rescale=1./255)

train_generator_cnn = train_datagen_cnn.flow_from_directory(
    TRAIN_DIR,
    target_size=(128, 128),
    batch_size=16,
    class_mode='binary'
)

test_generator_cnn = test_datagen_cnn.flow_from_directory(
    TEST_DIR,
    target_size=(128, 128),
    batch_size=16,
    class_mode='binary',
    shuffle=False
)

class_names = list(train_generator_resnet.class_indices.keys())
print(f"\nКлассы: {class_names}")
```

Листинг 3.3 — Продолжение листинга 3.2

```
base_model = ResNet50(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3)
)

base_model.trainable = False

x = GlobalAveragePooling2D()(base_model.output)
x = Dense(512, activation='relu')(x)
x = tf.keras.layers.Dropout(0.5)(x) # Регуляризация
output = Dense(1, activation='sigmoid')(x)

model = Model(inputs=base_model.input, outputs=output)

model.compile(
    optimizer=Adam(learning_rate=0.0001), # Низкий lr для
transfer learning
    loss='binary_crossentropy',
    metrics=['accuracy',
             tf.keras.metrics.Precision(name='precision'),
             tf.keras.metrics.Recall(name='recall')]
)

model.summary()

early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)

history = model.fit(
    train_generator_resnet,
    steps_per_epoch=train_generator_resnet.samples // BATCH_SIZE,
    validation_data=test_generator_resnet,
    validation_steps=test_generator_resnet.samples // BATCH_SIZE,
    epochs=20,
    callbacks=[early_stopping]
)
```

Листинг 3.4 — Продолжение листинга 3.3

```
# Сохранение модели
model.save('cataract_classification_resnet.h5')

# Загрузка случайного тестового изображения
test_images, test_labels = next(test_generator_resnet)
random_index = np.random.randint(0, len(test_images))

# Предсказание
pred = model.predict(np.expand_dims(test_images[random_index],
axis=0))
pred_class = class_names[int(pred > 0.5)]
true_class = class_names[int(test_labels[random_index])]

# Визуализация
plt.imshow(test_images[random_index])
plt.title(f"True: {true_class}\nPredicted: {pred_class} (prob:
{pred[0][0]:.2f})")
plt.axis('off')
plt.show()
```